

Lab 8 – More Objects and Inheritance

Aim

This week is focused on the object inheritance and the correct handling of overriding. This builds on your object work from Lab 6, extends it using inheritance and overriding, and calling its method in client class using polymorphism as discussed in Lecture 8.

Tips:

1. Complete the previous lab sheet before you start this sheet.
2. Ask a TA if you do not understand the lab tasks, or you cannot explain the output.

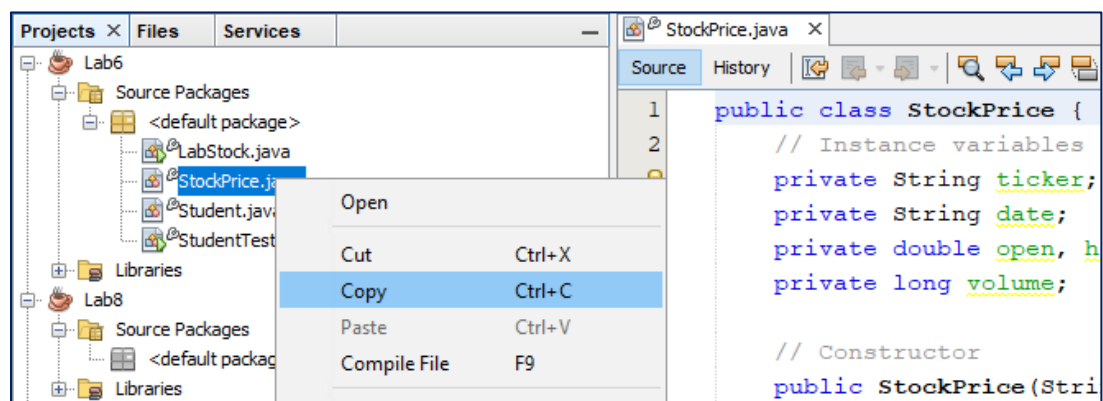
Lab Task 8.1 StockPricePlus

In Lab 6 you implemented the StockPrice class. However, this StockPrice object is very basic and lacks some features. Rather than changing StockPrice, we are going to use inheritance to create a new class.

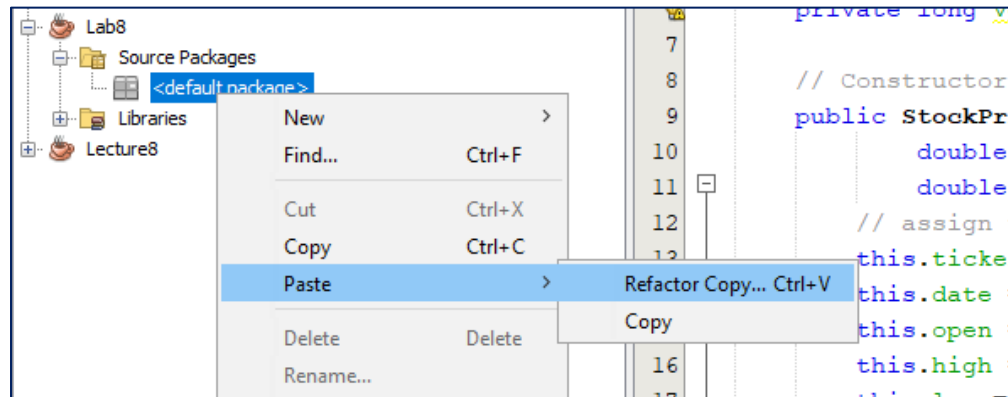
*Check whether your StockPrice class behaves as specified in Lab 6 before doing this lab!
Alternatively, you can also download the StockPrice.java in Learning Mall*

Follow these steps to implement a class called StockPricePlus:

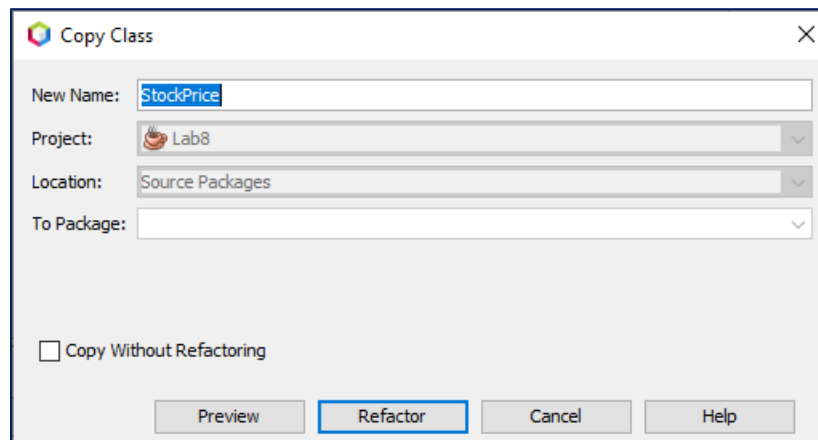
- Create a new project named Lab8.
- Copy the original StockPrice source file into your project.
 - Right click on the original StockPrice.java source file completed in Lab 6, click Copy.



- Right click <default package> folder in project Lab8 ; Click Paste -> Refactor Copy



- Click Refactor to copy the StockPrice.java source file into your new project.



- Create a new class named StockPricePlus, which is a subclass of StockPrice. (i.e., StockPricePlus extends StockPrice).
- Complete the constructor of StockPricePlus.
 - It should take the same number of parameters as its superclass, StockPrice.
 - Pass all values to the constructor of StockPrice using super().
- Create a new StockPricePlus object in main, and prints it.

As an example for such object, you may use:

```
StockPricePlus spp1 = new StockPricePlus("AAPL", "2006-12-1", 91.8, 92.33,  
90.1, 91.32, 28395700);
```

The output should look like:

```
Stock AAPL 2006-12-1 created.  
Stock: ticker=AAPL, date=2006-12-1, opening price=91.8, closing price=91.32, high=92.33, low=90.1, volume=28395700
```

- We are able to print the StockPricePlus object because StockPricePlus inherits the toString() method from the StockPrice class.
 - Recall that in Lab 6, we overrode toString() method which was inherited from its superclass, the Object class, to print the information of a StockPrice object.
- In the StockPricePlus class, add an instance variable numPrinted .
We want to use this new instance variable to record the number of times a StockPricePlus object has been printed using the toString() method.
 - Initialize it in the constructor to 0,
since when an object is newly created, it must have never been printed before.
- Now, override the toString() method in StockPricePlus
 - Correctly update the instance variable numPrinted to record how many times that particular object has been printed.
 - Don't forget to add the annotation @Override to make sure you are actually overriding the corresponding method in the superclass.
 - We also want to add this numPrinted information to the output string of toString().
Modify the method so that it prints that additional information as well,
and when we print the same object *multiple times*, we get:

```
Stock AAPL 2006-12-1 created.  
Stock: ticker=AAPL, date=2006-12-1, opening price=91.8,  
Printed 1 times  
Stock: ticker=AAPL, date=2006-12-1, opening price=91.8,  
Printed 2 times  
Stock: ticker=AAPL, date=2006-12-1, opening price=91.8,  
Printed 3 times
```

- You can use super. to call the overridden toString() method of the superclass.

- We want to further enhance `StockPricePlus` so that it can count and remember the total number of prints occurred for *all* `StockPricePlus` objects.
 - Add a class variable / static variable `totalPrinted` in the `StockPricePlus` class. Initialize it to 0 in the class.
 - Modify the overridden method `toString()` accordingly, to correctly update the new class variable as well.
- To obtain the total number of prints for all `StockPricePlus` objects, define a static getter method `getTotalPrinted()` to return the value of `totalPrinted`.
 - To solve this problem, you should review the difference between static and non-static instance variables and methods in Lecture 6 and Lecture 8.
- As a client class to test `getTotalPrinted()`, create a new class `StockPricePlusClient` with a `main` method.
 - Create the following three objects in `main`:
`StockPrice sp1 = new StockPrice("AAPL", "2006-12-1", 91.8, 92.33, 90.1, 91.32, 28395700);`
`StockPrice sp2 = new StockPricePlus("AAPL", "2006-12-4", 91.88, 92.05, 90.5, 91.12, 25340600);`
`StockPricePlus sp3 = new StockPricePlus("AAPL", "2006-12-5", 91.655, 92.33, 90.87, 91.27, 23672800);`
 - Then, print them a number of times, for example:
`System.out.println(sp1);`
`System.out.println(sp2);`
`System.out.println(sp3);`
`System.out.println(sp3);`
 - Call the static method `getTotalPrinted()`.
Recall how to call a static method from a client class in Week 4 and 5.
 - Discuss with your friends why the output on the above `println`s is as follows:

```
Stock AAPL 2006-12-1 created.
Stock AAPL 2006-12-4 created.
Stock AAPL 2006-12-5 created.
Stock: ticker=AAPL, date=2006-12-1, opening price=91.8, cl
Stock: ticker=AAPL, date=2006-12-4, opening price=91.88, c
Printed 1 times
Stock: ticker=AAPL, date=2006-12-5, opening price=91.655,
Printed 1 times
Stock: ticker=AAPL, date=2006-12-5, opening price=91.655,
Printed 2 times
Total StockPricePlus prints = 3
```

It has something to do with Polymorphism (especially for `sp2`), as discussed at the end of Lecture 8.

Lab Task 8.2 StockPrice Array

- Create an array of StockPrice in main method of StockPricePlusClient named stocks
 - Set the size to 3.
 - Set the array elements to the following objects:

```
StockPrice("AAPL", "2007-12-1", 91.8, 92.33, 90.1, 91.32, 28395700)
StockPricePlus("AAPL", "2007-12-4", 91.88, 93.05, 90.5, 91.12,
25340600)
StockPricePlus("AAPL", "2007-12-5", 91.655, 92.33, 90.87, 91.27,
23672800)
```

- Why can you set a StockPricePlus object as elements of a StockPrice array?
- In Lab 6, you have created getSpread() method that returns the spread of a StockPrice.

Now, create a static method `StockPrice getLargestSpread(StockPrice[] stocks)` in the `StockPricePlusClient` class to return the `StockPrice` object in the array which has the *largest spread*.

- Why can you call the `getSpread()` method of `StockPrice` on a `StockPricePlus` object?
- Call the `getLargestSpread` method in main and print the returned object having the largest spread.
- Discuss with your friends or your TA why the output is as follows:

```
Stock AAPL 2007-12-1 created.
Stock AAPL 2007-12-4 created.
Stock AAPL 2007-12-5 created.
Stock: ticker=AAPL, date=2007-12-4, opening price=91.88,
Printed 1 times
```

This is the end of CPT111-2324 Lab 8 Task Sheet.